How-To Library

Informatica

Integrating Data Engineering Integration on the AWS platform with Cloudera Data Platform (CDP) Public Cloud

© Copyright Informatica LLC 2020, 2023. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

Customers of Amazon Web Services and Informatica can integrate Data Engineering Integration 10.5.3 with a CDP compute cluster in the AWS cloud environment. The integration allows users to run mappings and workflows on CDP to access data from and write data to Delta Lake tables.

This article instructs administrators how to integrate Data Engineering Integration with CDP in the AWS environment.

Supported Versions

- Data Engineering Integration 10.5.3

Table of Contents

CDP Integration Overview. 2
Integrating DEI with CDP Public Cloud. 3
Before you begin CDP Public Cloud integration. 3
Read the Release Notes. 3
Verify System Requirements. 3
Configure Hadoop Clusters. 3
Configure Informatica Domain. 6
Creating a Cluster Configuration. 6
Importing a Hadoop Cluster Configuration from a File. 7
Creating a Hadoop Connection. 8
Configuring Hadoop Connection Properties. 8
Hadoop Connection - Cluster Environment Variables (Shared conref). 8
Cluster Library Path. 9
Common Advanced Properties. 9
Hadoop Connection - Blaze Advanced Properties (Shared conref). 9
Hadoop Connection -Spark Advanced Properties (Shared Conref). 10

CDP Integration Overview

Data Engineering Integration can connect to Cloudera Data Platform (CDP) Public Cloud on an AWS cloud environment. CDP Public Cloud is an integrated analytics and data management platform deployed on cloud services. It offers broad data analytics and artificial intelligence functionality along with secure user access and data governance features.

The Data Integration Service installs the binaries required to integrate an Informatica domain with the Hadoop environment. The integration requires Informatica connection objects and cluster configurations. A cluster configuration is a domain object that contains configuration parameters that you import from the Hadoop cluster. You then associate the cluster configuration with connections to access the Hadoop environment.

When you run a mapping, the Data Integration Service checks for the binary files on the cluster. If they do not exist or they are not synchronized, the Data Integration Service prepares the files for transfer. Data Integration Service transfers the files to the distributed cache through the Informatica Hadoop staging directory on HDFS. By default, the staging directory is /tmp. This transfer process replaces the requirement to install distribution packages on the Hadoop cluster.

Integrating DEI with CDP Public Cloud

You can integrate an Informatica domain with Hadoop clusters through Data Engineering Integration.

Perform the following tasks to integrate an Informatica domain with the Hadoop environment:

1. Install or upgrade to the current Informatica version.
2. Perform pre-import tasks, such as verifying system requirements and permissions. For more information, see the "Before you begin CDP Public Cloud integration" on page 3 section.
3. Import the cluster configuration into the domain. The cluster configuration contains properties from the *-site.xml files on the cluster. For more information, see the "Creating a Cluster Configuration" on page 6 section.
4. Create a Hadoop connection to run mappings within the Hadoop environment. For more information, see the "Creating a Hadoop Connection" on page 8 topic.
5. Perform post-import tasks specific to the Hadoop distribution that you integrate with.

Before you begin CDP Public Cloud integration

Perform the tasks in this section before you integrate Data Engineering Integration with CDP public cloud.

Read the Release Notes

Read the Release Notes for updates to the installation and upgrade process. You can also find information about known and fixed issues for the release.

Find the Release Notes on the Informatica documentation portal.

Verify System Requirements

Verify that your environment meets the following minimum system requirements for the integration:

Informatica services

Install and configure the Informatica services and the Developer tool. Verify that the domain contains a Model Repository Service and a Data Integration Service.

Domain access to CDP Public Cloud

Verify domain access to CDP through one of the following methods:

- The Informatica domain and CDP are installed on the same AWS VPS.
- The Informatica domain and CDP are installed on different AWS VPS's with peering enabled.

For more information about product requirements and supported platforms, see the Product Availability Matrix.

Configure Hadoop Clusters

Perform the following tasks in this section to configure the Hadoop cluster.

Create a Machine User

Create a machine user to provide programmatic access to CDP.

You can define the machine user account in Informatica to create and manage clusters and run jobs in CDP using the CLI or API commands. You must have a PowerUser role to create a machine user.

Perform the following steps to create a machine user in CDP.

1. Log in to the CDP web interface.
2. Navigate to **Management Console** > **User Management** > **Users**.
3. From the **Actions** menu, select **Create Machine User**.
4. Provide a name and click **Create**.
5. Provide access to all the services of the cluster.
6. Provide the Hive warehouse directory access to the user.

Create a keytab user

A keytab user is a user account that is used to authenticate with Kerberos services using a keytab file instead of a password. A keytab file contains the encrypted keys that are used for authentication with Kerberos services. When a user logs in using a keytab file, the Kerberos service uses the keys in the keytab to authenticate the user without requiring a password.

Perform the following steps to create a keytab user in CDP.

1. Open the CDP Management Console and navigate to the environment where you want to create the keytab user.
2. Select the **Users** tab, and click **Create User**.
3. Enter the username for the keytab user, and select **Keytab** as the authentication method.
4. Enter a password for the keytab user, and click **Create User**.
5. From the **Actions** menu drop-down list, select **Generate Keytabs**.

   The keytabs are generated and downloaded to your local machine. You can distribute the keytabs to the appropriate services as needed.

Configure system properties

Configure the following properties based on your usage:

1. If you want to access Avro tables, set the following property in `hive-site.xml`:
   - `metastore.storage.schema.reader.impl=org.apache.hadoop.hive.metastore.SerDeStorageSchemaReader`

     ```
     <property>
     <name>hive.default.rcfile.serde</name>
     <value>org.apache.hadoop.hive.serde2.columnar.ColumnarSerDe</value>
     </property>
     <property>
     <name>hive.default.rcfile.serde</name>
     <value>org.apache.hadoop.hive.serde2.columnar.ColumnarSerDe</value>
     </property>
     ```
   - `hive.support.concurrency – true`

     `hive.enforce.bucketing – true`

     `hive.exec.dynamic.partition.mode – nonstrict`

```
hive.txn.manager – org.apache.hadoop.hive.ql.lockmgr.DbTxnManager

hive.compactor.initiator.on – true

hive.compactor.worker.threads –1
```

2. If you are using Serde libraries, perform the following steps:
   a. Create the following folder structure in your Cloudera directory:

      ```
      /opt/cloudera/parcels/CDH/aux/jars/
      ```

   b. Copy the `hive-hcatalog-core-3.1.3000.7.2.12.3-2.jar` file from the following location:

      ```
      /opt/cloudera/parcels/CDH-7.2.12-1.cdh7.2.12.p3.20687891/lib/hive-hcatalog/share/hcatalog
      ```

   c. Paste the file in the following location:

      ```
      /opt/cloudera/parcels/CDH/aux/jars/
      ```

   d. Update the `Hive Auxiliary JARs Directory` property with the following path:

      ```
      /opt/cloudera/parcels/CDH/aux/jars/
      ```

3. If you are using HDFS file system, configure the following property to add the proxy entry for the keytab user:

   ```
   <property><name>hadoop.proxyuser.<keytab_user>.users</name><value><machine_user></
   value></property>
   ```

4. If you are using Amazon S3 and Redshift, add the following property in `Cluster-wide Advanced Configuration Snippet (Safety Valve) for core-site.xml`:

   ```
   fs.s3a.path.style.access=true
   ```

5. For external S3 bucket, add the following property in `Hadoop connection > Spark configuration > Advanced property`. Additionally, add the default CDP bucket name.

   ```
   spark.yarn.access.hadoopFileSystems=s3a://infarndawsdeepa7212/infarndawsdeepadl,s3a://
   s3shyalinirnd/
   ```

6. If you use Sqoop in Blaze engine, add the following property without any value in the `Cluster-wide Advanced Configuration Snippet (Safety Valve) for core-site.xml` and `HDFS Client Advanced Configuration Snippet (Safety Valve) for hdfs-site.xml` files:

   ```
   hadoop.security.key.provider.path
   ```

7. For Sqoop connector, add the following property in `CCO > core-site.xml` of Admin console.

   ```
   fs.AbstractFileSystem.hdfs.impl = org.apache.hadoop.fs.Hdfs
   ```

8. Provide permissions to submit yarn applications to the keytab and machine users.
9. Add proxy entries in HDFS and provide permissions in ranger and the keytab and machine users.
10. If you are running complex cases with date and time fields, add the following properties in Hadoop connection > Spark configuration > Advanced properties:

    ```
    spark.sql.legacy.avro.datetimeRebaseModeInRead LEGACY
    spark.sql.legacy.avro.datetimeRebaseModeInWrite LEGACY
    spark.sql.legacy.parquet.datetimeRebaseModeInRead LEGACY
    spark.sql.legacy.parquet.datetimeRebaseModeInWrite LEGACY
    spark.sql.legacy.parquet.int96RebaseModeInRead LEGACY
    spark.sql.legacy.parquet.int96RebaseModeInWrite LEGACY
    spark.sql.legacy.timeParserPolicy LEGACY
    ```

11. For Hbase configuration, perform the following steps:
    a. Add the following property in all the Hadoop connections:

       ```
       spark.yarn.access.hadoopFileSystems=<DefaultFS of datalake>
       ```

    b. To run Hbase mappings, copy the `hbase-site.xml` file from the datalake node and refresh the CCO with the file.

Note: When you are running mappings on the datalake location, change the HDFS storage to Datalake. You can find defaultFS of datalake in the `/etc/hadoop/conf/core-site.xml` file of the cluster under the fs.defaultFS property.

12. For streaming use cases, add the following properties:
    - For Hbase cases, add the following property in Hadoop Connection > Advanced Properties:

      ```
      spark.hbase.connector.security.credentials.enabled =false
      ```
    - For Blob cases, if you are using external table, configure the following property in `hive site.xml` of the CCO:

      ```
      hive.metastore.dml.events=false
      ```
    - For Blob cases, if you are using internal table, configure the following property in `hive site.xml` of the CCO:

      ```
      hive.metastore.dml.events=true
      ```

Configure Informatica Domain

Perform the following steps to configure the Informatica domain to run mappings on the CDP cluster:

1. Copy the `krb5.conf` file from the cluster to the domain.
2. Run the following command to fetch the IP address of the KDC server:

   ```
   ping kdc.<KDC_Hostname>
   ```
3. Add the IP address to the following properties in the realms section of the `krb5.conf` file:

   ```
   kdc = <KDC_IP_Address>
   admin_server = <KDC_IP_Address>
   ```
4. Change the default value of the UDP property to 1 in the server location of the `krb5.conf` file.
5. Add the cluster host entries to the virtual machine where the domain is installed.
6. To get the truststore file containing the certificates, go to `/etc/hadoop/conf/ssl-client.xml` in the CDP cluster.

   Truststore location is available in the `ssl.client.truststore.location` property. The password for the file is available in the `ssl.client.truststore.password` property.
7. Import the cluster certificates to the respective files in the domain.
8. Copy the `cm-auto-global_truststore.jks` file from the cluster folder `/var/lib/cloudera-scm-agent/agent-cert/` and paste it in the following domain VM folder:

   ```
   /var/lib/cloudera-scm-agent/agent-cert/
   ```

Creating a Cluster Configuration

After the Hadoop administrator prepares the cluster for import, the Informatica administrator must create a cluster configuration.

Perform this task in the following situations:
- You are integrating for the first time. - You upgraded from a previous version.

A cluster configuration is an object in the domain that contains configuration information about the Hadoop cluster. The cluster configuration enables the Data Integration Service to push mapping logic to the Hadoop environment. Import configuration properties from the Hadoop cluster to create a cluster configuration.

The import process imports values from *-site.xml files into configuration sets based on the individual *-site.xml files. When you perform the import, the cluster configuration wizard can create Hadoop, HBase, HDFS, and Hive connection to access the Hadoop environment. If you choose to create the connections, the wizard also associates the cluster configuration with the connections.

Note: If you are integrating for the first time and you imported the cluster configuration when you ran the installer, you *must* re-create or refresh the cluster configuration.

Importing a Hadoop Cluster Configuration from a File

You can import properties from an archive file to create a cluster configuration.

Before you import from the cluster, you must get the archive file from the Hadoop administrator.

1. From the **Connections** tab, click the **ClusterConfigurations** node in the Domain Navigator.
2. From the Actions menu, select **New** > **Cluster Configuration**.

   The **Cluster Configuration** wizard opens.
3. Configure the following properties:

Property	Description
Cluster configuration name	Name of the cluster configuration.
Description	Optional description of the cluster configuration.
Distribution type	The cluster Hadoop distribution type.
Method to import the cluster configuration	Choose **Import from file** to import properties from an archive file.
Create connections	Choose to create Hadoop, HDFS, Hive, and HBase connections. If you choose to create connections, the Cluster Configuration wizard associates the cluster configuration with each connection that it creates. The Hadoop connection contains default values for properties such as cluster environment variables, cluster path variables, and advanced properties. Based on the cluster environment and the functionality that you use, you can add to the default values or change the default values of these properties. For a list of Hadoop connection properties to configure, see "Configuring Hadoop Connection Properties" on page 8. If you do not choose to create connections, you must manually create them and associate the cluster configuration with them. **Important:** When the wizard creates the Hive connection, it populates the Metadata Connection String and the Data Access Connection String properties with the value from the hive.metastore.uris property. If the Hive metastore and HiveServer2 are running on different nodes, you must update the Metadata Connection String to point to the HiveServer2 host. When you create a Hive connection, the JDBC endpoint will be created with port 2182. Modify the port to 2181 to enable the clients to connect to Hive Zookeeper.

4. Click **Browse** to select a file. Select the file and click **Open**.
5. Click **Next** and verify the cluster configuration information on the summary page.

Creating a Hadoop Connection

Create a Hadoop connection before you run a mapping in the Hadoop environment.

1. Click **Window** > **Preferences**.
2. Select **Informatica** > **Connections**.
3. Expand the domain in the **Available Connections** list.
4. Select the **Cluster** connection type in the **Available Connections** list and click **Add**.

   The **New Cluster Connection** dialog box appears.
5. Enter the general properties for the connection.
6. Click **Next**.
7. Enter the Hadoop cluster properties, common properties, and the reject directory properties.
8. Click **Next**.
9. Click **Next**.

   Effective in version 10.2.2, Informatica dropped support for the Hive engine. Do not enter Hive configuration properties.
10. Enter configuration properties for the Blaze engine and click **Next**.
11. Enter configuration properties for the Spark engine and click **Finish**.

Configuring Hadoop Connection Properties

When you create a Hadoop connection, default values are assigned to cluster environment variables, cluster path properties, and advanced properties. You can add or edit values for these properties. You can also reset to default values.

You can configure the following Hadoop connection properties based on the cluster environment and functionality that you use:

- Cluster Environment Variables
- Cluster Library Path
- Common Advanced Properties
- Blaze Engine Advanced Properties
- Spark Engine Advanced Properties

Note: Informatica does not recommend changing these property values before you consult with third-party documentation, Informatica documentation, or Informatica Global Customer Support. If you change a value without knowledge of the property, you might experience performance degradation or other unexpected results.

To reset to default values, delete the property values. For example, if you delete the values of an edited Cluster Library Path property, the value resets to the default $DEFAULT_CLUSTER_LIBRARY_PATH.

Hadoop Connection - Cluster Environment Variables (Shared conref)

Cluster Environment Variables property lists the environment variables that the cluster uses. Each environment variable contains a name and a value. You can add environment variables or edit environment variables.

To edit the property in the text box, use the following format with &: to separate each name-value pair:

```
<name1>=<value1>[&:<name2>=<value2>…&:<nameN>=<valueN>]
```

Configure the following environment variables in the **Cluster Environment Variables** property:

HADOOP_NODE_JDK_HOME

Represents the directory from which you run the cluster services and the JDK version that the cluster nodes use. Required to run the Java transformation in the Hadoop environment and Sqoop mappings on the Blaze engine. Default is /usr/java/default. The JDK version that the Data Integration Service uses must be compatible with the JDK version on the cluster.

Set the property to <cluster JDK home>/jdk<version>.

For example, `HADOOP_NODE_JDK_HOME=<cluster JDK home>/jdk<version>`.

Cluster Library Path

Cluster Library Path property is a list of path variables for shared libraries on the cluster. You can add or edit library path variables.

To edit the property in the text box, use the following format with : to separate each path variable:

```
<variable1>[:<variable2>…:<variableN]
```

Configure the library path variables in the **Cluster Library Path** property.

Common Advanced Properties

Common advanced properties are a list of advanced or custom properties that are unique to the Hadoop environment. The properties are common to the Blaze and Spark engines. Each property contains a name and a value. You can add or edit advanced properties.

To edit the property in the text box, use the following format with &: to separate each name-value pair:

```
<name1>=<value1>[&:<name2>=<value2>…&:<nameN>=<valueN>]
```

Configure the following property in the **Advanced Properties** of the common properties section:

infapdo.java.opts

List of Java options to customize the Java run-time environment. The property contains default values.

If mappings in a MapR environment contain a Consolidation transformation or a Match transformation, change the following value:

- -Xmx512M. Specifies the maximum size for the Java virtual memory. Default is 512 MB. Increase the value to at least 700 MB.
  For example, `infapdo.java.opts=-Xmx700M`

Hadoop Connection - Blaze Advanced Properties (Shared conref)

Blaze advanced properties are a list of advanced or custom properties that are unique to the Blaze engine. Each property contains a name and a value. You can add or edit advanced properties.

To edit the property in the text box, use the following format with &: to separate each name-value pair:

```
<name1>=<value1>[&:<name2>=<value2>…&:<nameN>=<valueN>]
```

Configure the following properties in the **Advanced Properties** of the Blaze configuration section:

infagrid.cadi.namespace

Namespace for the Data Integration Service to use. Required to set up multiple Blaze instances.

Set the property to <unique namespace>.

For example, `infagrid.cadi.namespace=TestUser1_namespace`

infagrid.blaze.console.jsfport

JSF port for the Blaze engine console. Use a port number that no other cluster processes use. Required to set up multiple Blaze instances.

Set the property to <unique JSF port value>.

For example, `infagrid.blaze.console.jsfport=9090`

infagrid.blaze.console.httpport

HTTP port for the Blaze engine console. Use a port number that no other cluster processes use. Required to set up multiple Blaze instances.

Set the property to <unique HTTP port value>.

For example, `infagrid.blaze.console.httpport=9091`

infagrid.node.local.root.log.dir

Path for the Blaze service logs. Default is /tmp/infa/logs/blaze. Required to set up multiple Blaze instances.

Verify that all blaze users have write permission on /tmp.

Set the property to <local Blaze services log directory>.

For example, `infagrid.node.local.root.log.dir=<directory path>`

infacal.hadoop.logs.directory

Path in HDFS for the persistent Blaze logs. Default is /var/log/hadoop-yarn/apps/informatica. Required to set up multiple Blaze instances.

Set the property to <persistent log directory path>.

For example, `infacal.hadoop.logs.directory=<directory path>`

infagrid.node.hadoop.local.root.log.dir

Path in the Hadoop connection for the service log directory.

Set the property to <service log directory path>.

For example, `infagrid.node.local.root.log.dir=$HADOOP_NODE_INFA_HOME/blazeLogs`

Hadoop Connection -Spark Advanced Properties (Shared Conref)

Spark advanced properties are a list of advanced or custom properties that are unique to the Spark engine. Each property contains a name and a value. You can add or edit advanced properties. Each property contains a name and a value. You can add or edit advanced properties.

Configure the following properties in the **Advanced Properties** of the Spark configuration section:

To edit the property in the text box, use the following format with &: to separate each name-value pair:

`<name1>=<value1>[&:<name2>=<value2>…&:<nameN>=<valueN>]`

infasjs.env.spark.context-settings.passthrough.spark.dynamicAllocation.executorIdleTimeout

Maximum time that an Spark Jobserver executor node can be idle before it is removed. Increase the value to assist in debugging data preview jobs that use the Spark engine.

You can specify the time in seconds, minutes, or hours using the suffix *s*, *m*, or *h*, respectively. If you do not specify a time unit, the property uses milliseconds.

If you disable dynamic resource allocation, this property is not used.

Default is 120s.

infasjs.env.spark.jobserver.max-jobs-per-context

Maximum number of Spark jobs that can run simultaneously on a Spark context. If you increase the value of this property, you might need to allocate more resources by increasing *spark.executor.cores* and *spark.executor.memory*.

Default is 10.

infasjs.env.spark.jobserver.sparkJobTimeoutInMinutes

Maximum time in minutes that a Spark job can run on a Spark context before the Spark Jobserver cancels the job. Increase the value to assist in debugging data preview jobs that use the Spark engine.

Default is 15.

infaspark.class.log.level.map

Logging level for specific classes in the Spark driver or executor. When you configure this property, it overrides the tracing level you set for the mapping.

Set the value of this property to a JSON string in the following format: {"<fully qualified class name":"<log level>"}

Join multiple class logging level statements with a comma. You can use the following logging levels: FATAL, WARN, INFO, DEBUG, ALL.

For example, set the property to:

```
infaspark.class.log.level.map={"org.apache.spark.deploy.yarn.ApplicationMaster":"TRACE","
org.apache.spark.deploy.security.HadoopFSDelegationTokenProvider":"DEBUG"}
```

infaspark.driver.cluster.mode.extraJavaOptions

List of extra Java options for the Spark driver that runs inside the cluster. Required for streaming mappings to read from or write to a Kafka cluster that uses Kerberos authentication.

For example, set the property to:

```
infaspark.driver.cluster.mode.extraJavaOptions=
-Djava.security.egd=file:/dev/./urandom
-XX:MaxMetaspaceSize=256M -Djavax.security.auth.useSubjectCredsOnly=true
-Djava.security.krb5.conf=/<path to keytab file>/krb5.conf
-Djava.security.auth.login.config=<path to jaas config>/kafka_client_jaas.config
```

To configure the property for a specific user, you can include the following lines of code:

```
infaspark.driver.cluster.mode.extraJavaOptions =
-Djava.security.egd=file:/dev/./urandom
-XX:MaxMetaspaceSize=256M -XX:+UseG1GC -XX:MaxGCPauseMillis=500
-Djava.security.krb5.conf=/etc/krb5.conf
```

infaspark.driver.log.level

Logging level for the Spark driver logs. When you configure this property, it overrides the tracing level you set for the mapping.

Set the value to one of the following levels: FATAL, WARN, INFO, DEBUG, ALL.

infaspark.executor.extraJavaOptions

List of extra Java options for the Spark executor. Required for streaming mappings to read from or write to a Kafka cluster that uses Kerberos authentication.

For example, set the property to:

```
infaspark.executor.extraJavaOptions=
-Djava.security.egd=file:/dev/./urandom
```

```
-XX:MaxMetaspaceSize=256M -Djavax.security.auth.useSubjectCredsOnly=true
-Djava.security.krb5.conf=/<path to krb5.conf file>/krb5.conf
-Djava.security.auth.login.config=/<path to jAAS config>/kafka_client_jaas.config
```

To configure the property for a specific user, you can include the following lines of code:

```
infaspark.executor.extraJavaOptions =
-Djava.security.egd=file:/dev/./urandom
-XX:MaxMetaspaceSize=256M -XX:+UseG1GC -XX:MaxGCPauseMillis=500
-Djava.security.krb5.conf=/etc/krb5.conf
```

infaspark.executor.log.level

Logging level for the Spark executor logs. When you configure this property, it overrides the tracing level you set for the mapping.

Set the value to one of the following levels: FATAL, WARN, INFO, DEBUG, ALL.

infaspark.flatfile.writer.nullValue

When the Databricks Spark engine writes to a target, it converts null values to empty strings (" "). For example, 12, AB,"",23p09udj.

The Databricks Spark engine can write the empty strings to string columns, but when it tries to write an empty string to a non-string column, the mapping fails with a type mismatch.

To allow the Databricks Spark engine to convert the empty strings back to null values and write to the target, configure the property in the Databricks Spark connection.

Set the property to: TRUE

infaspark.json.parser.mode

Specifies the parser how to handle corrupt JSON records. You can set the value to one of the following modes:

- DROPMALFORMED. The parser ignores all corrupted records. Default mode.
- PERMISSIVE. The parser accepts non-standard fields as nulls in corrupted records.
- FAILFAST. The parser generates an exception when it encounters a corrupted record and the Spark application goes down.

infaspark.json.parser.multiLine

Specifies whether the parser can read a multiline record in a JSON file. You can set the value to true or false. Default is false. Applies only to non-native distributions that use Spark version 2.2.x and above.

infaspark.pythontx.exec

Required to run a Python transformation on the Spark engine for Data Engineering Integration. The location of the Python executable binary on the worker nodes in the Hadoop cluster.

For example, set the property to:

```
infaspark.pythontx.exec=/usr/bin/python3.4
```

If you use the installation of Python on the Data Integration Service machine, set the value to the Python executable binary in the Informatica installation directory on the Data Integration Service machine.

For example, set the property to:

```
infaspark.pythontx.exec=INFA_HOME/services/shared/spark/python/lib/python3.4
```

infaspark.pythontx.executorEnv.LD_PRELOAD

Required to run a Python transformation on the Spark engine for Data Engineering Streaming. The location of the Python shared library in the Python installation folder on the Data Integration Service machine.

For example, set the property to:

```
infaspark.pythontx.executorEnv.LD_PRELOAD=
INFA_HOME/services/shared/spark/python/lib/libpython3.6m.so
```

infaspark.pythontx.executorEnv.PYTHONHOME

Required to run a Python transformation on the Spark engine for Data Engineering Integration and Data Engineering Streaming. The location of the Python installation directory on the worker nodes in the Hadoop cluster.

For example, set the property to:

```
infaspark.pythontx.executorEnv.PYTHONHOME=/usr
```

If you use the installation of Python on the Data Integration Service machine, use the location of the Python installation directory on the Data Integration Service machine.

For example, set the property to:

```
infaspark.pythontx.executorEnv.PYTHONHOME=
INFA_HOME/services/shared/spark/python/
```

infaspark.pythontx.submit.lib.JEP_HOME

Required to run a Python transformation on the Spark engine for Data Engineering Streaming. The location of the Jep package in the Python installation folder on the Data Integration Service machine.

For example, set the property to:

```
infaspark.pythontx.submit.lib.JEP_HOME=
INFA_HOME/services/shared/spark/python/lib/python3.6/site-packages/jep/
```

infaspark.useHiveWarehouseAPI

Enables the Hive Warehouse Connector. Set the property to TRUE.

For example, `infaspark.useHiveWarehouseAPI=true`.

spark.authenticate

Enables authentication for the Spark service on Hadoop. Required for Spark encryption.

Set the property to TRUE.

For example, `spark.authenticate=TRUE`

spark.authenticate.enableSaslEncryption

Enables encrypted communication when SASL authentication is enabled. Required if Spark encryption uses SASL authentication.

Set the property to TRUE.

For example, `spark.authenticate.enableSaslEncryption=TRUE`

spark.datasource.hive.warehouse.load.staging.dir

Directory for the temporary HDFS files used for batch writes to Hive. Required when you enable the Hive Warehouse Connector.

For example, set to `/tmp`

spark.datasource.hive.warehouse.metastoreUri

URI for the Hive metastore. Required when you enable the Hive Warehouse Connector. Use the value for *hive.metastore.uris* from the hive_site_xml cluster configuration properties.

For example, set the value to `thrift://mycluster-1.com:9083 .`

spark.driver.cores

Indicates the number of cores that each driver uses uses to run jobs on the Spark engine.

Set to: `spark.driver.cores=1`

spark.driver.extraJavaOptions

List of extra Java options for the Spark driver.

When you write date/time data within a complex data type to a Hive target using a Hortonworks HDP 3.1 cluster, append the following value to the property: `-Duser.timezone=UTC`

spark.driver.memory

Indicates the amount of driver process memory that the Spark engine uses to run jobs.

Recommended value: Allocate at least 256 MB for every data source.

Set to: `spark.driver.memory=3G`

spark.executor.cores

Indicates the number of cores that each executor process uses to run tasklets on the Spark engine.

Set to: `spark.executor.cores=1`

spark.executor.extraJavaOptions

List of extra Java options for the Spark executor.

When you write date/time data within a complex data type to a Hive target using a Hortonworks HDP 3.1 cluster, append the following value to the property: `-Duser.timezone=UTC`

spark.executor.instances

Indicates the number of instances that each executor process uses to run tasklets on the Spark engine.

Set to: `spark.executor.instances=1`

spark.executor.memory

Indicates the amount of memory that each executor process uses to run tasklets on the Spark engine.

Set to: `spark.executor.memory=3G`

spark.hadoop.hive.llap.daemon.service.hosts

Application name for the LLAP service. Required when you enable the Hive Warehouse Connector. Use the value for *hive.llap.daemon.service.hosts* from the hive_site_xml cluster configuration properties.

spark.hadoop.hive.zookeeper.quorum

Zookeeper hosts used by Hive LLAP. Required when you enable the Hive Warehouse Connector. Use the value for *hive.zookeeper.quorum* from the hive_site_xml cluster configuration properties.

spark.datasource.hive.warehouse.load.staging.dir

Directory for the temporary HDFS files used for batch writes to Hive. Required when you enable the Hive Warehouse Connector.

For example, set the property to `/tmp`

spark.hadoop.validateOutputSpecs

Validates if the HBase table exists. Required for streaming mappings to write to a HBase target in an Amazon EMR cluster. Set the value to false.

spark.scheduler.maxRegisteredResourcesWaitingTime

The number of milliseconds to wait for resources to register before scheduling a task. Default is 30000. Decrease the value to reduce delays before starting the Spark job execution. Required to improve performance for mappings on the Spark engine.

Set the property to 15000.

For example, `spark.scheduler.maxRegisteredResourcesWaitingTime=15000`

spark.scheduler.minRegisteredResourcesRatio

The minimum ratio of registered resources to acquire before task scheduling begins. Default is 0.8. Decrease the value to reduce any delay before starting the Spark job execution. Required to improve performance for mappings on the Spark engine.

Set the property to: 0.5

For example, `spark.scheduler.minRegisteredResourcesRatio=0.5`

spark.shuffle.encryption.enabled

Enables encrypted communication when authentication is enabled. Required for Spark encryption.

Set the property to TRUE.

For example, `spark.shuffle.encryption.enabled=TRUE`

spark.sql.hive.hiveserver2.jdbc.url

URL for HiveServer2 Interactive. Required to use the Hive Warehouse Connector. Use the value in Ambari for HiveServer2 JDBC URL.

spark.yarn.access.hadoopFileSystems

Comma-separated list of external file systems that the Spark service can access. By default, the Spark service has access to the file systems listed in fs.defaultFS in the core-site.xml configuration set of the cluster configuration. Set this property to give the Spark service access to additional file systems.

If you run a mapping on a Cloudera CDP Public Cloud cluster and you use an HDFS on a Cloudera Data Lake cluster, you must allow access to that file system. Append the value for the property *fs.defaultFS* found in `core-site.xml` on the Data Lake cluster. For example: `spark.yarn.access.hadoopFileSystems=hdfs://infarndcdppamdl-master1.infarndc.src9-ltfl.cloudera.site:8020`

Author

Dipchandan Kalita